School of Computer Science

FACULTY OF ENGINEERING AND PHYSICAL SCIENCES

UNIVERSITY OF LEEDS

Final Report

Machine Learning Classification Model of Chess Playing Styles trained using Game Data

Matthew Hughes

Submitted in accordance with the requirements for the degree of MEng, BSc Computer Science

2025/2026

COMP3931 Individual Project

The candidate confirms that the following have been submitted*:*

Items	**Format**	**Recipient(s) and Date**
Final Report	*PDF file*	*Uploaded to Minerva (29/04/26)*
Link to online code repository	*https://github.com/mattyh867/chess-playing-style-classifier*	*Sent to supervisor and assessor (29/04/26)*
Webpage	*https://chess-playing-style-classifier-production-ee70.up.railway.app/*	*(29/04/26)*

The candidate confirms that the work submitted is their own and the appropriate credit has been given where reference has been made to the work of others.

I understand that failure to attribute material which is obtained from another source may be considered as plagiarism.

_____ Matthew Hughes

© 2026 The University of Leeds and Matthew Hughes

Summary

High-level chess players have had access to coaches and personalised analysis for decades, while the average player can only rely on general guidance, such as books and videos, that cannot directly advise based on the individual. The goal of this project is to provide a tool that can be used by intermediate players to give them a better idea of their playing style, allowing them to seek resources suited to their style. This project develops a machine learning pipeline that assigns one of four playing styles (aggressive, positional, defensive, balanced) to a chess game based on features extracted from game data.

The dataset was built from 2015 Lichess games played at 1500 Elo rating and above, with 16 features extracted per game using the Stockfish engine at depth 12. Labels were assigned automatically via a rule-based heuristic grounded in chess theory. Three classifiers were trained and evaluated: a Random Forest Classifier (91.2%), a Feed-Forward Neural Network (89.8%), and Extreme Gradient Boosting Classifier (96.9%). The performance ordering is consistent with Grinsztajn et al. (2022), who found tree-based models tend to outperform neural networks on structured tabular data. Feature importance analysis across all three models consistently highlighted captures per move, prophylactic moves, and positional sacrifices as the strongest discriminators.

Cross-skill evaluation was conducted on 2016 Lichess data across two Elo brackets, and a web interface was developed to make the system accessible to non-expert users, satisfying all five project objectives. Limitations include the absence of an external ground truth for labelling validation, and known misclassification of historical games where Stockfish penalises principled sacrificial play as errors.

Acknowledgements

I would like to thank my supervisor, Fahad Panolan, for their guidance, feedback, and continued support throughout the duration of this project. I would also like to thank my assessor, Dibyayan Chakraborty, for the constructive feedback provided during the progress meeting.

Table of Contents

Summary ..... **iii**
Acknowledgements ..... **iv**
Table of Contents ..... **v**
Chapter 1 Introduction **1**
1.1 Outline 1
Chapter 2 Background Research **2**
2.1 Playing Style Classification 2
2.2 Previous Clustering and Classification Approaches 3
2.3 Feature Engineering 3
2.4 Feature Design for Style Classification 4
2.5 Learned Representation 4
2.6 Practical Considerations 5
Chapter 3 Methodology **6**
3.1 Overview of Methodology 6
3.2 Project Management 7
3.3 Data Collection and Preparation Pipeline 8
3.3.1 Data Source 8
3.3.2 Feature Extraction 9
3.4 Random Forest Classifier 10
3.5 Feed-Forward Neural Network 12
3.6 Extreme Gradient Boosting Classifier 14
Chapter 4 Implementation and Validation **16**
4.1 Dataset Construction and Labelling Pipeline 16
4.1.1 Engine Configuration and Feature Extraction 16
4.1.2 Labelling and Threshold Calibration 17
4.1.3 Batch Processing 17
4.2 Random Forest Classifier 18
4.2.1 Data Loading and Feature Selection 18
4.2.2 Train-Test Split and Preprocessing 18
4.2.3 Model Configuration and Training 18
4.2.4 Evaluation 19

4.2.5 Artifact Preservation 19
4.3 Feed-Forward Neural Network 19
4.3.1 Data Preparation 20
4.3.2 Model Architecture 20
4.3.3 Training Procedure 20
4.3.4 Evaluation 21
4.3.5 Artifact Preservation 21
4.4 Extreme Gradient Boosting Classifier 21
4.4.1 Data Loading and Feature Selection 21
4.4.2 Train-Test Split and Preprocessing 22
4.4.3 Model Configuration and Training 22
4.4.4 Evaluation 22
4.4.5 Artifact Preservation 23
Chapter 5 Results, Evaluation and Discussion 24
5.1 Evaluation Methodology 24
5.2 In-Distribution Classification Performance 24
5.2.1 Random Forest Classifier 24
5.2.2 Feed-Forward Neural Network 25
5.2.3 Extreme Gradient Boosting Classifier 25
5.3 Model Comparison 25
5.4 Feature Importance Analysis 26
5.5 Cross-Skill Generalisation 27
5.6 Limitations and Failure Cases 28
5.7 Conclusions 29
5.8 Future Work 30
List of References 31
Appendix A Self-appraisal 32
A.1 Critical self-evaluation 32
A.2 Personal reflection and lessons learned 32
A.3 Legal, social, ethical and professional issues 33
A.3.1 Legal issues 33
A.3.2 Social issues 33
A.3.3 Ethical issues 33
A.3.4 Professional issues 33

Appendix B External Materials..... 34

Chapter 1
Introduction

1.1 Outline

This project aims to apply machine learning techniques to identify and classify distinct playing styles in chess from historical game data. Chess players exhibit remarkably different approaches to the game - some prefer aggressive attacks and material sacrifices, while others favour patient manoeuvring and solid defensive structures. However, analysing a player's style typically requires manual review of many games by experienced players, making it time-intensive and subjective.

The computer science challenge lies in teaching a computer to recognise these qualitative patterns automatically. Unlike simpler problems such as predicting who will win a game (based on ratings or positions), style classification requires the system to understand *how* players make decisions, not just whether those decisions are objectively good. The computer must learn to extract meaningful patterns from the complexity of chess where billions of possible positions exist and the "best" move often depends on context rather than fixed rules. This involves designing algorithms that can analyse thousands of games, identify consistent behavioural patterns across different situations, and classify them into distinct style categories, all while handling the challenge that the same player might employ different styles in different games.

The goal is to create a practical tool that allows average chess players to understand their own playing tendencies without needing expert analysis or deep technical knowledge. This project addresses a real-world need by making style analysis accessible, moving beyond existing chess tools that focus primarily on finding the strongest moves rather than understanding human playing patterns.

Chapter 2
Background Research

The challenge of automatically identifying chess playing styles has been approached from several angles in prior research, each with distinct methodologies and limitations.

2.1 Playing Style Classification

Chess players exhibit distinct strategic approaches that extend beyond finding the strongest individual moves. These playing styles represent fundamental differences in how players conceptualise and execute their game plans, reflecting different risk tolerances, positional philosophies, and decision-making preferences.

Aggressive play prioritises seizing initiative and creating immediate tactical threats, often at the cost of material equality. Players adopting this style actively seek sharp positions with complex tactics, frequently initiating early attacks on the opponent's king. The aggressive approach is characterised by a willingness to sacrifice material for compensation such as development advantages or exposed king positions. From an analytical perspective, aggressive play manifests through elevated rates of checks, captures, material sacrifices, and early attacking moves.

Positional play emphasises accumulating long-term strategic advantages through superior pawn structure, piece placement, and space control rather than immediate tactical strikes. This style is characterised by prophylactic thinking, where moves prevent opponent plans while incrementally improving one's position. Positional players favour strategic over material sacrifices and consistently find moves that maintain or improve positional evaluation without creating immediate threats. Notable practitioners include Anatoly Karpov and Tigran Petrosian. Computationally, this style appears through elevated rates of prophylactic moves, positional sacrifices, and best move accuracy.

Defensive play represents a reactive rather than proactive strategic approach, prioritising solid positions and risk minimisation. Players employing this style focus on neutralising opponent threats through simplification, piece trades, and counterattacking from stable positions. When facing pressure, defensive players characteristically trade pieces to reduce attacking potential, retreat from aggressive squares, and make moves that consolidate rather than expand. Mathematically, defensive patterns manifest through high rates of

simplifications, retreating moves, trades when material down, and passive moves that maintain evaluation without active engagement.

Balanced play does not represent a single coherent philosophy but rather the absence of a dominant strategic tendency. Players classified as balanced exhibit no strong signals across any one category of behaviour; they make captures at moderate rates without consistently seeking material gain, find strong engine moves at reasonable frequency without prioritising accuracy above all else, and neither retreat habitually nor commit to sustained attacking plans. This versatility may reflect genuine adaptability - adjusting approach to the demands of each position - or simply an inconsistency in strategic decision-making. Computationally, balanced games are identified by the absence of a sufficiently high score in any single style category, with no feature ratio reaching the threshold required to assign a definitive label and are treated as the default classification when no dominant pattern emerges.

2.2 Previous Clustering and Classification Approaches

Jayasekara (2018) demonstrated that distinct playing styles in chess are computationally differentiable through machine learning techniques. By analysing games from top-level players, the research showed that clustering methods could successfully group games according to playing styles, with classification algorithms including Logistic Regression, Naïve Bayes, and Random Forest achieving accurate mappings between extracted features and predetermined style categories. This work established the feasibility of automated style classification and showed that Random Forest classifiers are particularly effective for this task. However, Jayasekara's approach relied on assumed playing styles for professional players rather than developing a systematic labelling methodology and did not address how such systems could be deployed for average players seeking to understand their own styles.

2.3 Feature Engineering

When applying machine learning to chess style analysis, a fundamental challenge is determining the characteristics in a game that are meaningful in relation to playing styles. Feature engineering aims to extract quantifiable attributes from raw game data and is central to addressing this challenge.

Traditional chess engines rely heavily on hand-crafted evaluation functions developed by experts. Silver et al. (2018) noted that "The strongest programs are based on a combination of sophisticated search techniques, domain-specific adaptations, and handcrafted evaluation

functions that have been refined by human experts over several decades.” These systems required chess masters to manually encode positional concepts such as centre control, king safety, and pawn structure into explicit, quantifiable features. While effective for engine strength, this approach presents limitations for style classification, as it is applied in the context of objective evaluation over behavioural characterisation. However, despite the different use case, there still exists a subset of features from the ones used in chess engines that can provide playing style insights.

2.4 Feature Design for Style Classification

Research specifically targeting playing style has developed features that characterise decision-making tendencies rather than position quality. A predictive modelling study extracted eight distinct features to differentiate styles among elite players including Magnus Carlsen, Hikaru Nakamura, and Garry Kasparov (Kalagara, 2024). These features ranged from game length (distinguishing patient players who prefer lengthy strategic battles from those seeking quick tactical victories) to trade frequency (capturing willingness to simplify positions versus maintaining tension). The research successfully trained models to distinguish between players, establishing that behavioural features could serve as effective style discriminators.

A key difference to observe here is that the mentioned research focuses on five selected players, all of whom are well-known grandmasters at the very upper echelons of chess skill, while this project uses games from random players to attempt to create a more relatable model.

2.5 Learned Representation

The development of the chess engine AlphaZero marked a paradigm shift away from hand-crafted features toward learned representations. Unlike traditional engines requiring explicit feature engineering, AlphaZero achieved superhuman performance "starting from random play, and given no domain knowledge except the game rules" (Silver et al., 2018). Through self-play and deep reinforcement learning, it automatically discovered relevant positional concepts without human intervention, eliminating the need for explicit feature design.

As with AlphaZero, learned representation approaches in chess have generally targeted optimal play rather than human style characterisation; the Maia Chess project, however,

represents a notable exception. McIlroy-Young et al. (2020) adapted AlphaZero's deep learning architecture specifically for modelling human behaviour, training separate neural networks on games from players at different rating levels. Rather than hand-crafting features, Maia learned representations directly from millions of human games, achieving significantly higher accuracy in predicting human moves than traditional engines. This demonstrated that neural networks could capture subtle behavioural patterns without explicit feature engineering, though at the cost of model interpretability.

2.6 Practical Considerations

This project must weigh the implications of each approach. Hand-crafted feature engineering offers interpretability, as the behaviour patterns distinguishing different playing styles remain identifiable. Learned representations with neural networks may provide better accuracy and detect more subtle behaviour patterns not captured by expert-designed features; however, this introduces a 'black box' in which the learned patterns are not directly interpretable.

A further consideration is computational power. This project was conducted on limited hardware compared to that of other projects such as the Maia Chess project, meaning some accuracy may need to be sacrificed to keep the project within a reasonable scope, and the results of certain approaches are affected by this processing constraint. For example, analysing games using a chess engine at a higher depth would grant access to a more accurate set of best possible moves but would take significantly longer. Similarly, incorporating more games into the preprocessing pipeline would yield more reliable results but would exceed the time available for this project.

Chapter 3
Methodology

This chapter describes the approach taken to develop a machine learning-based chess playing style classifier that enables the average chess player to understand their general playing style. The methodology addresses the core requirement of creating an accessible tool rather than a complex game analysis system. The approach was structured into five phases, each with defined deliverables aligned to the project objectives established in Chapter 1.

3.1 Overview of Methodology

The project is split into 5 distinct stages of development: data preprocessing, training the Random Forest classifier, training the Feed-Forward Neural Network and XGBoost classifier, model evaluation and comparison, and publication of the resulting tools.

A phased approach allows time and resources to be efficiently allocated. It minimises the refactoring of code compared to a non-structured project plan and arranges the development process such that each stage produces a deliverable and addresses an objective of the project.

Stage 1 focuses on the first objective, which is to develop and validate a labelled dataset on which the models can be trained. This stage includes the design and implementation of domain-specific features to increase model accuracy.

Stages 2 and 3 are centred around the development of the three classification models: Random Forest, Feedforward Neural Network, and XGBoost. XGBoost was added as a third model to broaden the comparative analysis with a gradient boosting approach, which operates on fundamentally different ensemble principles to the other two methods. Each model utilises the labelled data from the previous stage to produce a classifier capable of identifying a user's playing style from a game.

Stage 4 consists of evaluating and comparing the three models based on performance metrics such as accuracy, precision, and F1 score. This stage also considers which model is most suitable for the average player, and which should be recommended based on different hardware constraints.

Stage 5 delivers the products of all previous stages through an accessible medium that allows users to input a chess game that they have played and receive a chess playing style.

GitHub is the chosen version control tool used throughout the project.

3.2 Project Management

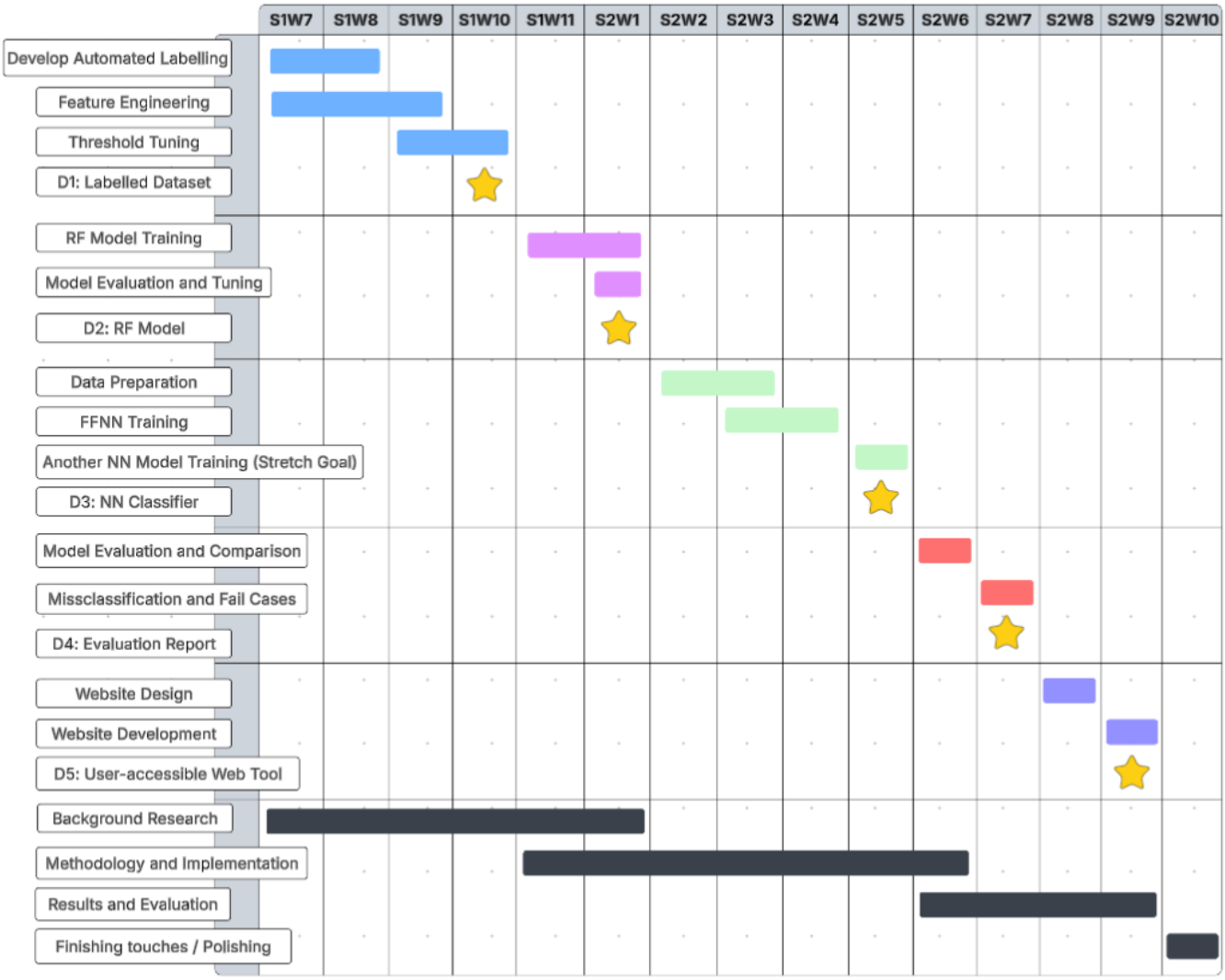


Figure 3.2 A Gantt Chart showing the project plan over two semesters.

This project adopted a milestone-driven development methodology, structured around five deliverables (D1–D5) that correspond directly to the five project objectives. This approach was chosen over a waterfall model because the outputs of each phase directly inform the design decisions of the next; for example, feature engineering from the labelling phase is also used in the training of the Random Forest model. Version control was maintained throughout using Git, with the project repository hosted on GitHub. All source code and

processing scripts were committed incrementally, providing a full trail of development decisions and ensuring reproducibility of all results. Figure 3.2 shows the rough estimates of each stage of development.

3.3 Data Collection and Preparation Pipeline

3.3.1 Data Source

Data preparation and validation is the first phase of the project, ensuring that a reliable and usable dataset is available before model development begins. This ordering was chosen because subsequent phases depend directly on the structure and quality of the data; had the training procedure been designed first, a dataset would still have been required to evaluate it, making data preparation the logical starting point.

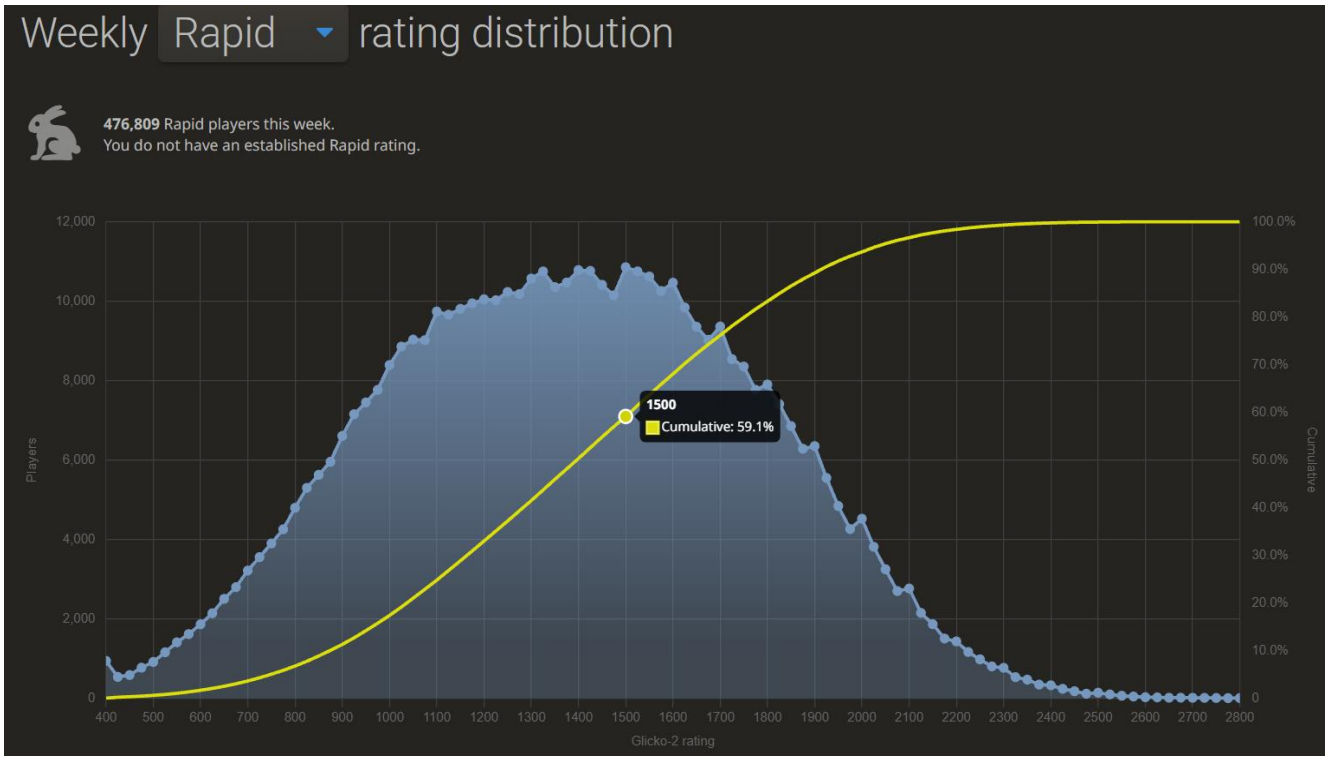


Figure 3.3 A graph showing the weekly rating distribution of the Lichess Open Database from 15th January 2026.

The Lichess Open Database of chess games provides large sets of games from players of all skill ratings from each year. The move sequence and skill rating of each player were extracted from games taken from a single year, up to a maximum of 10000 games, yielding 20000 labelled items. A minimum rating of 1500 was applied so that only games involving players at or above that threshold were labelled; games where a player of at least 1500 faces a player below 1500 yield only one labelled item. This minimum was chosen to eliminate games containing many blunders or mistakes, as such moves are more

representative of skill limitation than playing style. This threshold also reduces noise: a lower-skilled player might make an unintentional prophylactic move which would otherwise be detected via feature engineering and cause the algorithm to label that player's style as 'positional', when the move was not made with positional intent. Lastly, ratings of above 1500 represents the top 40.9% of all players on Lichess, as shown in Figure 3.3, making it a reasonable threshold when developing a tool intended for the average player. Although these statistics are from 2026, the Glicko-2 rating system ensures that the median rating does not deviate significantly from 1500 (Glickman, 1995).

Balanced class distribution was established as a design goal. However, achieving perfect balance proved difficult due to the inherently aggressive nature of chess at the 1500+ rating level, where players more frequently make captures and attacks than passive or defensive moves. Threshold tuning was applied iteratively to reduce class imbalance as much as possible while preserving the behavioural validity of each label.

3.3.2 Feature Extraction

Feature engineering is a vital part of the preprocessing pipeline. A total of 18 distinct features were defined, each extractable from a game with the assistance of a chess engine, Stockfish.

Stockfish was selected as the chess engine for position evaluation due to its widespread adoption in chess research and its status as the strongest open-source engine available, ensuring that position evaluations are both reliable and reproducible by other researchers.

Each game was analysed move-by-move and for each position, the engine evaluated the board state before and after the move, enabling identification of move characteristics such as evaluation changes, threats created, and positional improvements. Specific algorithms detected prophylactic play by finding moves that maintained evaluation without creating immediate threats or checks. Defensive patterns were recognised through retreat detection (pieces moving out of attacks) and passive move identification (moves with minimal evaluation change and no aggressive characteristics).

Labels were assigned to each game via rule-based thresholding applied to the extracted feature set, rather than through manual annotation or unsupervised clustering. This approach was chosen for scalability across a large dataset and to ensure that labels reflect consistent, reproducible criteria rather than subjective human judgement.

One feature that was considered but ultimately not used as a metric to classify any playing style was the opening choice. While there is no doubt that certain openings might be indications of the nature of the game about to be played, such as the French defence often

leading to a closed game that favours a more positional player, there were some reasons why opening choice is not used as a feature.

Firstly, a player's choice of opening can be influenced by factors unrelated to their playing style. For example, a player might have chosen the London opening because they saw it online and wanted to try it in an online game or perhaps a friend had recommended the Caro-Kann defence and they wanted to experiment. Secondly, openings may theoretically favour a playing style, but this might not always match the player using it. If a player opts for the French defence, they could end up playing it aggressively instead of positionally by going for unexpected early attacks. Additionally, at lower ratings, there is usually less intention when deciding what opening to use. It is less of a conscious philosophical choice about how they want to play the game and more of a memorisation of moves to get them through the early game.

3.4 Random Forest Classifier

Model training comprises the next phase of development. The decision to separate the models into individual development phases was guided by the characteristics of each approach. The Random Forest model was selected to be trained first because it is a more interpretable method than the neural network, meaning that the reasoning behind individual classifications can be more readily examined and validated.

Random Forest was selected as the initial classification model due to its suitability for structured feature data of the kind produced by the Stockfish analysis pipeline. As an ensemble method that aggregates predictions across many decision trees trained on bootstrap samples of the training data, Random Forest is inherently resistant to overfitting and performs well on moderate-sized datasets without requiring extensive hyperparameter optimisation (Breiman, 2001). For this project, the decision to use Random Forest first is also because it produces interpretable feature importance rankings, enabling identification of the behavioural features that contribute most to classification decisions. This interpretability is valuable both for validating that the model is learning meaningful patterns and for informing discussion about which aspects of chess behaviour most strongly distinguish playing styles.

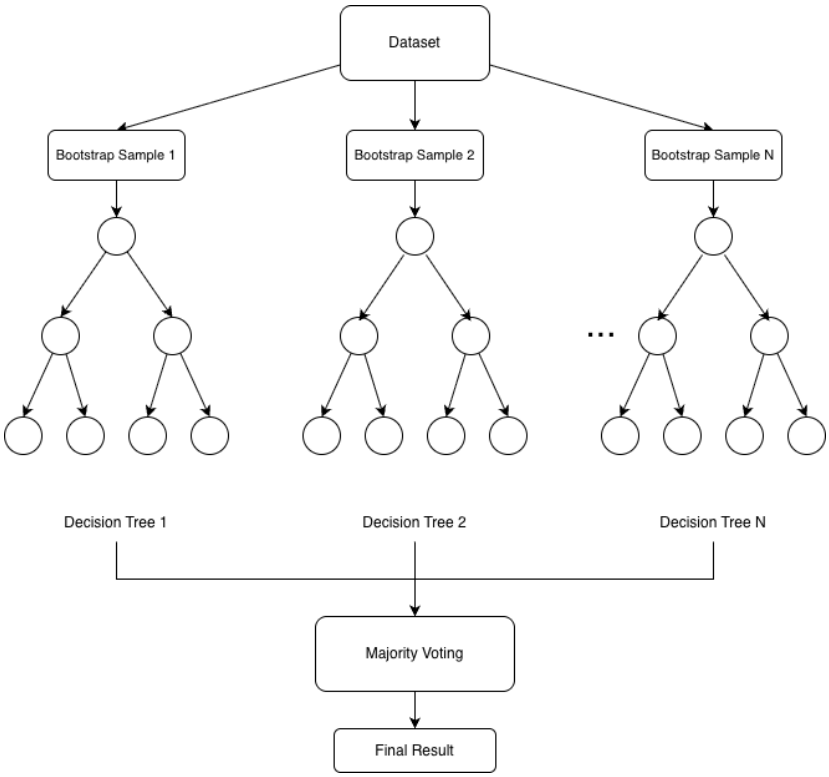


Figure 3.4 Architectural overview of the Random Forest classifier, showing how the dataset is used to train N independent decision trees whose outputs are combined via majority voting to produce a final playing style classification.

The dataset was divided into an 80% training set and 20% test set using stratified sampling to maintain class balance across both partitions. Stratification is particularly important for this problem because the four classes are not perfectly balanced; without it, the minority class (balanced) could be underrepresented in either partition, leading to unreliable evaluation. Although Random Forest does not strictly require feature scaling due to its use of decision boundaries rather than distance-based calculations, normalisation was applied to maintain consistency with the neural network pipeline and ensure that both models received identically preprocessed input.

Evaluation was conducted using per-class precision, recall, and F1-score in addition to overall accuracy. Per-class metrics were chosen over accuracy alone because in a four-class classification problem, aggregate accuracy can mask poor performance on individual classes. A confusion matrix was also generated to identify specific misclassification patterns between styles. Feature importance rankings were extracted from the trained model to identify which of the behavioural features extracted during the preprocessing stage contribute most to classification decisions.

Upon completion, three artifacts were saved to ensure reproducibility and enable deployment: the trained model serialised using joblib, the fitted StandardScaler for consistent preprocessing of new data, and the ordered list of feature column names to ensure correct

input structure. These artifacts enable the web interface to load the trained model and apply identical preprocessing to user-submitted games without retraining.

3.5 Feed-Forward Neural Network

To provide a comparison against the Random Forest baseline, a Feedforward Neural Network (FFNN) was developed as the second classification model. While the Random Forest was selected for its interpretability, a neural network was chosen for its capacity to learn non-linear relationships between features that tree-based models may not capture as effectively. The comparison between these two approaches is central to the project's evaluation, as it allows assessment of whether a more complex model architecture yields improved classification performance on chess behavioural data.

The same labelled dataset and feature set used to train the Random Forest were used for the FFNN to ensure a fair comparison between the two models. Unlike Random Forest, neural networks are sensitive to the magnitude of input values, making feature normalisation a requirement rather than an optional consistency measure as it was for the Random Forest.

A three-way train/validation/test split was used rather than the two-way split applied to the Random Forest. This is necessary because the FFNN requires a held-out validation set during training to monitor generalisation performance and to inform early stopping - a mechanism that has no equivalent in the Random Forest training procedure.

A fully connected feedforward architecture with two hidden layers of progressively smaller size was selected as appropriate for the scale and structure of the feature data. ReLU was chosen as the activation function for its computational efficiency and status as the standard default in modern feedforward networks. Dropout regularisation was applied after each hidden layer to reduce overfitting, which is a particular concern given the relatively small dataset size.

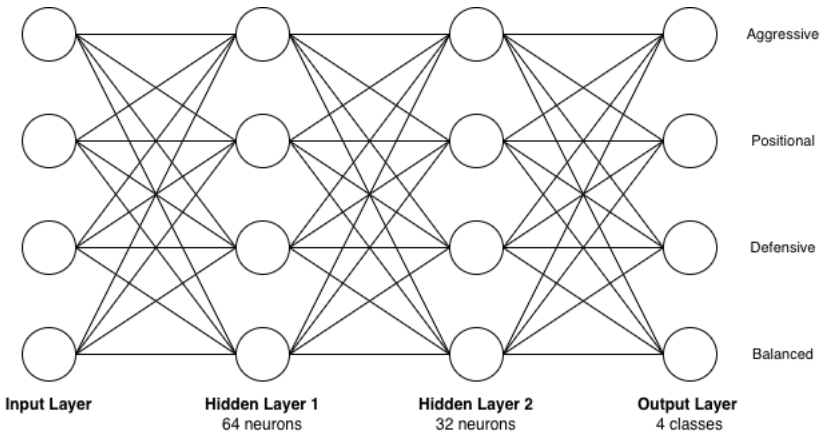


Figure 3.5 Architecture of the feedforward neural network, showing the input layer (16 features), two fully connected hidden layers with ReLU activation and Dropout regularisation, and the output layer producing a probability distribution over the four playing style classes.

The Adam optimiser was selected over standard SGD for its adaptive learning rate behaviour, which generally produces faster convergence on small tabular datasets (Kingma and Ba, 2014). It is also the standard default optimiser for feedforward networks, adapting the learning rate per parameter based on gradient history. Cross-entropy loss was used as the objective function, which is standard for multi-class classification problems. Early stopping was implemented to halt training when validation accuracy ceased to improve, ensuring the final model reflects peak generalisation performance rather than the state at the last epoch.

The trained model was evaluated using the same metrics as the Random Forest - overall accuracy, per-class precision, recall, and F1-score - to enable direct comparison between the two approaches. These results are discussed in Chapter 5 alongside the Random Forest and XGBoost results.

Upon completion, the trained model weights, fitted StandardScaler, and ordered feature column list were saved to ensure reproducibility and enable deployment, consistent with the artefact preservation approach taken for the Random Forest. These artefacts, alongside those produced by the Random Forest and XGBoost pipelines, provide everything required to load any of the three trained models and apply them to new data without retraining.

3.6 Extreme Gradient Boosting Classifier

To further broaden the comparative analysis, an XGBoost (Extreme Gradient Boosting) classifier was implemented as a third model alongside the Random Forest and FFNN. Unlike Random Forest, which builds trees independently in parallel, XGBoost is a gradient boosting method that constructs trees sequentially, with each tree trained to correct the residual errors of the previous ones (Chen and Guestrin, 2016). The objective function combines a standard classification loss with a regularisation term that penalises model complexity, providing a built-in mechanism for controlling overfitting that differs fundamentally from both the bootstrap aggregation used by Random Forest and the dropout regularisation used by the FFNN. XGBoost was selected because it represents a meaningfully different ensemble strategy while still operating on the same tabular feature data, making it a valuable addition to the comparative analysis.

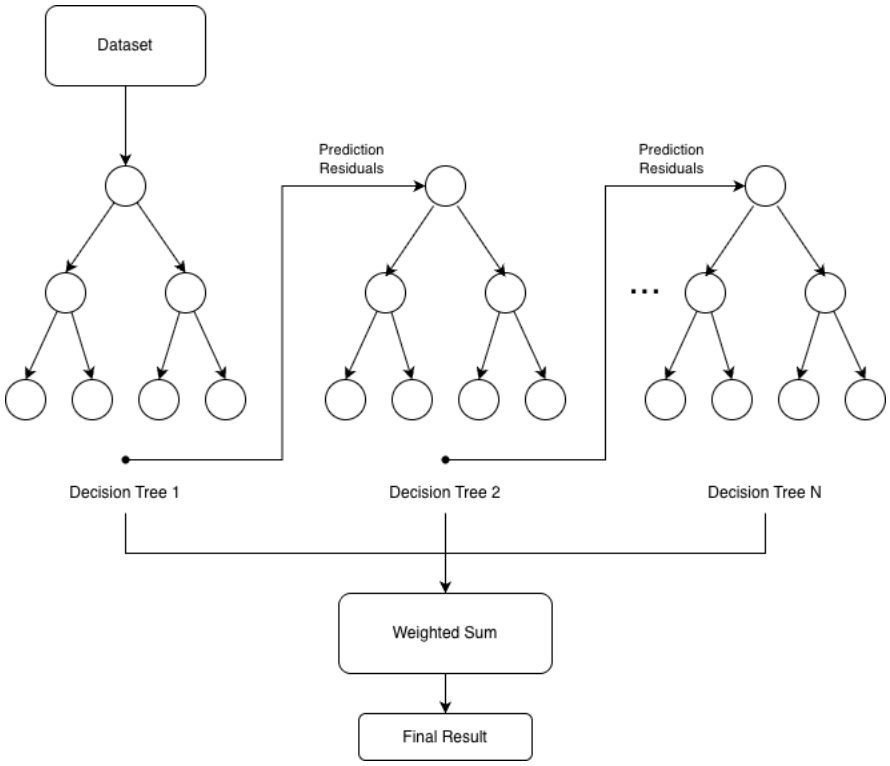


Figure 3.6 Architectural overview of the XGBoost classifier, showing how each decision tree is trained sequentially on the residual errors of the previous tree, with outputs combined via weighted sum to produce a final playing style classification.

The same labelled dataset and feature set used for the Random Forest and FFNN were used to train the XGBoost model, ensuring that all three models receive identical input and can be evaluated on a consistent basis. The dataset was divided into an 80% training set and 20% test set using stratified sampling to preserve class balance across both partitions, consistent with the Random Forest split. Although XGBoost does not require a validation set in the same way the FFNN does, a three-way train/validation/test split was applied to ensure the test set remained entirely unseen during training. The dataset was divided into 72% training, 8% validation, and 20% test partitions.

Normalisation was applied for consistency with the other models, though gradient boosting methods are not sensitive to feature scaling in the way neural networks are.

The model configuration was selected as a reasonable starting point based on common practice for tabular classification tasks rather than through systematic hyperparameter search, consistent with the approach taken for the Random Forest.

Evaluation was conducted using the same metrics as the other two models - overall accuracy, per-class precision, recall, and F1-score - alongside a confusion matrix to identify misclassification patterns between playing style classes. Feature importance scores were also extracted to compare which behavioural features XGBoost weighted most heavily against those identified by the Random Forest. These results are discussed in Chapter 5 alongside those of the Random Forest and FFNN.

Chapter 4
Implementation and Validation

This chapter describes the construction and validation of the three core system components developed to address the project objectives: the automated dataset labelling pipeline, the Random Forest classifier, and the two comparative models - a feedforward neural network and an XGBoost classifier. Each component was implemented iteratively, with validation conducted at each stage to confirm correct behaviour before proceeding. The chapter documents the key design decisions made during implementation, the challenges encountered and how they were resolved, and the testing performed to establish confidence in each deliverable.

4.1 Dataset Construction and Labelling Pipeline

The first phase produced the labelled dataset on which all three models are trained. The pipeline is implemented across two modules - chess_analyzer.py and processing.py - handling game analysis and batch processing respectively.

4.1.1 Engine Configuration and Feature Extraction

The ChessGameAnalyzer class wraps Stockfish, configured with a 4096MB hash table and 8 processing threads to maximise cache usage and reduce processing time. Analysis was performed at depth 12, which provides reliable positional evaluation while remaining computationally feasible given the hardware constraints discussed in Section 2.6. A shallower depth would introduce noise into centipawn loss calculations; deeper analysis was not practical for the volume of data.

For each player's turn, the engine evaluates the position before and after the move is played. The difference gives the centipawn loss for that move, which is accumulated and averaged across the game. The engine's principal variation before each move is also retrieved to determine whether the player found the top engine choice, contributing to the best_moves_found count. Mate scores are mapped to ±10,000 centipawns to prevent undefined behaviour during arithmetic.

Eighteen features are extracted per player per game across five categories: aggression (checks_given, captures_made, material_sacrifices, early_attacks), positional (prophylactic_moves, positional_sacrifices), defensive (simplifications, defensive_moves, counterattacks, retreat_moves, trades_when_losing, passive_moves), tactical

(avg_centipawn_loss, best_moves_found, tactical_shots, blunders), and general (complex_positions). Of these, 16 are passed to the classification models as input features; king_safety_risks and complex_positions are computed by the pipeline but were not incorporated into either the labelling system or the classifier, representing candidates for inclusion in future work. Each feature implements a behavioural concept described in Section 2.1; for example, a material sacrifice is recorded when a higher-value piece captures a lower-value piece, yet evaluation does not exceed a 50 centipawn change, and a prophylactic move is recorded when a quiet move produces a small positive evaluation shift without constituting a direct threat.

During deployment via the web interface, analysis depth was reduced to 6 to ensure processing completed within the server timeout window. While shallower than the depth used during training, depth 6 provides sufficient evaluation reliability for the feature types used.

4.1.2 Labelling and Threshold Calibration

The PlaystyleLabeler assigns one of four labels - aggressive, positional, defensive, or balanced - using a scoring mechanism rather than independent hard thresholds. Each feature ratio is compared against a calibrated threshold, with features that meet their threshold contributing a point score to the corresponding style's total. The style with the highest score is assigned, provided it reaches a minimum of 2.0; games where no style exceeds this floor are labelled balanced. Centipawn loss thresholds are adjusted dynamically by Elo rating to avoid confusing high accuracy with positional style in stronger players.

Arriving at the final thresholds required iterative calibration. Initial values produced severe class imbalance - approximately 53% aggressive and 3.5% defensive - because aggression thresholds were insufficiently discriminating and the defensive feature set lacked the retreat_moves, trades_when_losing, and passive_moves indicators. Both issues were addressed: the three defensive features were added, and aggression thresholds were raised to require a stronger behavioural signal. The pipeline was re-run after each adjustment until the distribution stabilised across all four classes without any single class dominating.

4.1.3 Batch Processing

BatchGameProcessor applies the pipeline at scale, filtering out games where either player falls below the 1500 rating minimum or the game is shorter than 20 moves, as very short games do not yield meaningful behavioural ratios. Results are saved at configurable intervals to guard against data loss during long runs.

Feature distributions within each class were examined to confirm that they separated in expected directions, for instance, that the aggressive class exhibited a higher mean captures-per-move than the positional class, providing confidence that the labeller captured genuine behavioural differences.

4.2 Random Forest Classifier

The Random Forest classifier was implemented in rf.py using scikit-learn's RandomForestClassifier (Pedregosa et al., 2011). The implementation follows directly from the design decisions described in Section 3.4, translating the architectural choices into a working training and evaluation pipeline.

4.2.1 Data Loading and Feature Selection

The labelled dataset produced by the preprocessing pipeline was loaded from a CSV file, with the metadata columns - including player name, Elo rating, colour, game ID, date, result, and opening - excluded from the feature set. This ensures the model learns from behavioural patterns extracted during Stockfish analysis rather than incidental game metadata. The remaining 16 columns formed the input feature matrix, with the label column as the classification target.

4.2.2 Train-Test Split and Preprocessing

The dataset was split into an 80% training set and 20% test set using stratified sampling with a fixed random seed of 42 for reproducibility, as described in Section 3.4.

Although Random Forest does not require feature scaling - its decision boundaries are invariant to monotonic transformations of the input - StandardScaler normalisation was applied to the training data and used to transform the test set. This was done to maintain consistency with the neural network and XGBoost pipelines, ensuring that all three models receive identically preprocessed input and that any observed performance differences are attributable to model architecture rather than data preparation.

4.2.3 Model Configuration and Training

The model was implemented using scikit-learn's RandomForestClassifier with 100 estimators, unlimited tree depth, and otherwise default parameters, which were found to produce strong classification performance without the need for extensive hyperparameter tuning. All available CPU cores were utilised during training via the n_jobs=-1 parameter to minimise computation time.

Class weighting was applied via scikit-learn's class_weight='balanced' parameter, which scales each class's influence on the model inversely proportional to its frequency in the training data. This was applied to compensate for the underrepresentation of the balanced class, which accounts for approximately 6.9% of the dataset.

The choice of 100 trees balances ensemble diversity against training time. Unlimited tree depth allows each tree to grow until all leaves are pure or contain a single sample, which is appropriate given Random Forest's inherent resistance to overfitting through bootstrap aggregation. Another protective measure against overfitting is having each split only consider a random subset of features, further diversifying the trees and reducing correlation between them. No further grid search or cross-validation was performed; the default configuration was treated as a reasonable baseline, consistent with the project's design rationale of establishing an interpretable benchmark before exploring more complex architectures.

4.2.4 Evaluation

The trained model was evaluated on the held-out test set using overall accuracy, per-class precision, recall, and F1-score, alongside a confusion matrix to identify systematic misclassification patterns between playing style classes. Feature importance scores were extracted from the fitted model to examine which behavioural features contributed most to classification decisions, providing a mechanism for validating that the model is learning meaningful chess-specific patterns rather than noise.

4.2.5 Artifact Preservation

Upon completion, three artifacts were saved to ensure reproducibility and enable deployment: the trained model serialised using joblib, the fitted StandardScaler for consistent preprocessing of new data, and the ordered list of feature column names to ensure correct input structure. These artifacts enable the web interface to load the trained model and apply identical preprocessing to user-submitted games without retraining.

4.3 Feed-Forward Neural Network

The FFNN was implemented across two modules: nnprep.py, which handles data preparation, and ffnn.py, which defines the model architecture, training loop, and evaluation pipeline. The implementation uses PyTorch rather than scikit-learn due to the greater flexibility it provides in defining custom architectures and training procedures, as described in Section 3.5.

4.3.1 Data Preparation

Data preparation is handled separately in nnprep.py to keep preprocessing logic independent of the training script. The same labelled dataset used for the Random Forest was loaded, with the same metadata columns excluded, ensuring both models receive identical input features.

The dataset was divided into three stratified partitions: 70% for training, 10% for validation, and 20% for testing, with a fixed random seed of 42 for reproducibility. A separate validation split is required for the FFNN - unlike the Random Forest, which requires no monitoring during training - because it is used to track generalisation performance each epoch and to inform early stopping decisions.

StandardScaler normalisation was applied by fitting exclusively on the training set and transforming the validation and test sets using those parameters, preventing any information leakage from held-out data. Each partition was then wrapped in a PyTorch TensorDataset and loaded into a DataLoader with a batch size of 32, enabling mini-batch gradient descent during training. The fitted scaler and feature metadata were saved as artifacts at this stage to ensure consistency when preprocessing new data during inference.

4.3.2 Model Architecture

The network is defined as the ChessStyleFFNN class, a fully connected feedforward architecture with two hidden layers. The input layer accepts the 16-feature vector produced by the Stockfish analysis pipeline. The first hidden layer maps this to 64 neurons, followed by a second hidden layer reducing to 32 neurons, and finally an output layer with 4 neurons corresponding to the four playing style classes. ReLU activation functions are applied after each hidden layer, and dropout regularisation at a rate of 0.3 is applied after each activation, as described in Section 3.5. No activation function is applied to the output layer, as PyTorch's CrossEntropyLoss criterion incorporates a softmax internally.

4.3.3 Training Procedure

The model was trained using the Adam optimiser with a learning rate of 0.001 and CrossEntropyLoss as the loss function. Class weights were computed using scikit-learn's compute_class_weight utility and passed directly to the CrossEntropyLoss criterion, increasing the penalty for misclassifying underrepresented classes during backpropagation. This was applied for the same reason as the Random Forest - to prevent the model from disproportionately optimising for the majority class.

Training ran for a maximum of 100 epochs, with early stopping applied using a patience of 15; if validation accuracy did not improve for 15 consecutive epochs, training was halted and

the best-performing model state was restored. This prevents unnecessary computation and avoids overfitting to the training set in later epochs.

Each epoch consisted of a training phase, where gradients were computed and weights updated via backpropagation, followed by a validation phase run under torch.no_grad() to compute validation loss and accuracy without updating parameters. Training and validation metrics were recorded each epoch to produce a learning curve, providing a visual diagnostic for convergence behaviour and potential overfitting.

The device was set dynamically (CUDA if a compatible GPU was available, otherwise CPU) ensuring the script could run on different hardware without modification.

4.3.4 Evaluation

The trained model was evaluated on the held-out test set using the same metrics as the Random Forest: overall accuracy, per-class precision, recall, and F1-score, and a confusion matrix. This ensures a consistent basis for comparison between the two models, with any performance differences attributable to architecture rather than evaluation methodology.

4.3.5 Artifact Preservation

Three artifacts were saved upon completion: the trained model weights serialised as a .pth file using torch.save, the training history as a pickle file containing per-epoch loss and accuracy for both training and validation sets, and a metadata file recording the final test accuracy, training timestamp, and architectural configuration. The scaler and feature column list saved during data preparation complete the set of artifacts needed to reproduce predictions on new data.

4.4 Extreme Gradient Boosting Classifier

The XGBoost classifier was implemented in xgb.py using the xgboost library's XGBClassifier, following directly from the design decisions described in Section 3.6. The implementation closely mirrors the structure of the Random Forest pipeline in rf.py, with adaptations specific to the gradient boosting framework.

4.4.1 Data Loading and Feature Selection

The same labelled dataset used for the Random Forest and FFNN was loaded from the CSV file produced by the preprocessing pipeline. The same metadata columns were excluded, leaving the identical 16 behavioural feature columns as the input feature matrix with the label column as the classification target, ensuring all three models are evaluated on identical input.

4.4.2 Train-Test Split and Preprocessing

The dataset was first split into an 80% training partition and 20% test set using stratified sampling with a fixed random seed of 42. A further 10% was then held out from the training partition as a validation set, resulting in an effective split of 72% training, 8% validation, and 20% test. Stratification was applied at both split stages to preserve class balance across all three partitions.

StandardScaler normalisation was applied by fitting on the training data and transforming the test set using those parameters to prevent data leakage. As noted in Section 3.6, XGBoost is not inherently sensitive to feature scaling, but normalisation was applied for consistency across all three pipelines.

XGBoost requires integer class labels rather than string labels, so a LabelEncoder was applied to map the four playing style strings - aggressive, positional, defensive, balanced - to numeric indices. The encoder was fitted on this fixed class ordering to ensure consistent label mapping during evaluation and inference.

4.4.3 Model Configuration and Training

The model was configured with 200 estimators, a maximum tree depth of 6, a learning rate of 0.1, and row and column subsampling rates of 0.8. These values were selected as a reasonable starting point based on common practice for tabular classification tasks, consistent with the rationale described in Section 3.6. No systematic grid search or cross-validation was performed. Multiclass log loss was specified as the evaluation metric, and all available CPU cores were utilised via n_jobs=-1 to minimise training time.

Sample weights were computed using scikit-learn's compute_sample_weight utility and passed to the sample_weight parameter of the fit method, as XGBoost does not expose a class_weight parameter directly for multiclass problems. This achieves the same effect as the class weighting applied to the other two models, ensuring consistent handling of class imbalance across all three pipelines.

The validation set was passed to XGBoost's eval_set parameter, allowing the model's error to be tracked on unseen data after each boosting round as a diagnostic for overfitting. This ensures the test set remains entirely unseen during training, maintaining methodological consistency with the Random Forest and FFNN pipelines.

4.4.4 Evaluation

The trained model was evaluated on the held-out test set using overall accuracy, per-class precision, recall, and F1-score, and a confusion matrix - the same metrics used for the

Random Forest and FFNN. Predicted numeric labels were decoded back to playing style strings using the LabelEncoder prior to computing all metrics, ensuring results are directly comparable with the string-labelled outputs of the other two models. The confusion matrix was additionally rendered as a percentage-annotated heatmap and saved to models/XGB/ as a visual diagnostic. Feature importance scores were extracted using XGBoost's built-in gain-based ranking and compared against the Random Forest's impurity-based rankings to examine whether the two ensemble methods identified the same behavioural features as most discriminating.

4.4.5 Artifact Preservation

Four artifacts were saved upon completion: the trained model serialised using joblib, the fitted StandardScaler, the ordered list of feature column names, and the fitted LabelEncoder. The inclusion of the LabelEncoder as a saved artifact - not required by the Random Forest or FFNN pipelines - reflects the additional label encoding step specific to XGBoost and is necessary to correctly decode predictions on new data during inference.

Chapter 5
Results, Evaluation and Discussion

5.1 Evaluation Methodology

Each of the three classifiers was evaluated on a held-out test set comprising 20% of the labelled dataset, with no overlap between training and test data across any model. Evaluation metrics include overall accuracy, per-class precision, recall, F1-score, and a confusion matrix. Per-class metrics were prioritised over accuracy alone because in a four-class problem, aggregate accuracy can mask poor performance on individual classes - particularly the underrepresented balanced class. All three models were trained and evaluated on identical feature sets and test splits, ensuring that performance differences are attributable to architecture rather than data.

A second evaluation was conducted to assess generalisation across skill levels, addressing Objective 4. Two held-out datasets were constructed from the 2016 Lichess Open Database - distinct from the 2015 training data - comprising 500 game records from players rated below 1400 and 500 from players rated above 2000. No ground truth labels were available for these sets; evaluation is therefore based on predicted class distributions and cross-model agreement, interpreted against domain knowledge.

5.2 In-Distribution Classification Performance

5.2.1 Random Forest Classifier

The Random Forest achieved an overall accuracy of 91.2% on the held-out test set. Aggressive achieved the highest F1-score of 93.6% (precision 92.7%, recall 94.5%), which is expected given that aggression-related features - captures_per_move and checks_per_move in particular - produce the clearest decision boundaries in the feature space. Positional and balanced achieved F1-scores of 91.0% and 89.8% respectively, while defensive was the weakest class at 87.6%.

The confusion matrix identifies defensive misclassifications as the primary source of error: 61 defensive samples were predicted as aggressive and 46 as positional. This is consistent with the overlapping feature signatures of reactive and cautious play - a player who retreats frequently and avoids material exchange shares observable characteristics with a passive

positional player. This aggressive-defensive boundary proves the hardest to resolve across all three models.

5.2.2 Feed-Forward Neural Network

The FFNN achieved an overall accuracy of 89.8%, approximately 1.4 percentage points below the Random Forest. This gap is consistent with findings reported by Grinsztajn et al. (2022), who demonstrated that tree-based models systematically outperform neural networks on tabular datasets with heterogeneous features. The behavioural feature set used here - combining count-based ratios, engine-derived statistics, and move-type indicators - fits this profile, making the RF advantage the expected outcome.

The most notable pattern is the behaviour of the balanced class: recall reaches 97.1% but precision falls to 75.9%, meaning the model over-predicts balanced and draws in misclassified positional and defensive cases. This reflects the tendency of neural networks to default toward lower-confidence catch-all categories near decision boundaries. Defensive is again the weakest class at 84.9% F1, with 74 defensive samples misclassified as aggressive - a higher count than in either the Random Forest or XGBoost confusion matrices.

5.2.3 Extreme Gradient Boosting Classifier

XGBoost achieved the highest overall accuracy of 96.9%, with all four classes recording F1-scores above 94%. The confusion matrix shows a marked reduction in aggressive-defensive confusions: only 19 aggressive samples were predicted as defensive, compared to 61 in the Random Forest and 74 in the FFNN. XGBoost's sequential tree construction trains each subsequent estimator on the residual errors of the previous ones, concentrating learning effort on the harder class boundaries that earlier trees misclassify. The built-in L1 and L2 regularisation in the objective function also prevents overfitting on the smaller balanced class in a way that dropout regularisation alone does not achieve for the FFNN. Both properties contribute directly to XGBoost's stronger performance on the classes where the other two models struggle most.

5.3 Model Comparison

The results for all three models are summarised in Table 5.1.

Model	Accuracy	Aggressive F1	Positional F1	Defensive F1	Balanced F1
RF	91.2%	93.6%	91.0%	87.6%	89.8%
FFNN	89.8%	93.0%	90.6%	84.9%	85.2%
XGBoost	**96.9%**	**98.1%**	**97.7%**	**94.5%**	**96.6%**

Table 5.1 Per-class F1-scores and overall accuracy for all three classifiers on the held-out test set.

The 1.4 percentage point gap between RF and FFNN is consistent with the theoretical expectation from Grinsztajn et al. (2022). The 5.7 point gap between XGBoost and the Random Forest is more substantial and reflects the difference in ensemble strategy: where Random Forest aggregates independently constructed trees, XGBoost iteratively refines predictions by targeting previously misclassified samples - a mechanism that disproportionately benefits the defensive class where both other models exhibit the most confusion. All three models substantially exceed chance-level performance of 25%, confirming that the behavioural features extracted by the preprocessing pipeline carry genuine discriminative signal. No comparison against external benchmarks is made, as the labelling has no external ground truth.

5.4 Feature Importance Analysis

Feature importance rankings were extracted from both the Random Forest (impurity-based) and XGBoost (gain-based) models and compared to examine whether two architecturally distinct approaches converge on the same behavioural signals. The top features shared by both models include captures_per_move, prophylactic_moves, positional_sacrifices, and counterattacks. This alignment is meaningful: captures and checks are the primary signals for the aggressive class, prophylactic moves and positional sacrifices are definitionally linked to positional play, and counterattacks are a core marker of the defensive label. The convergence of these rankings across both models provides stronger evidence of genuine feature relevance than either ranking alone.

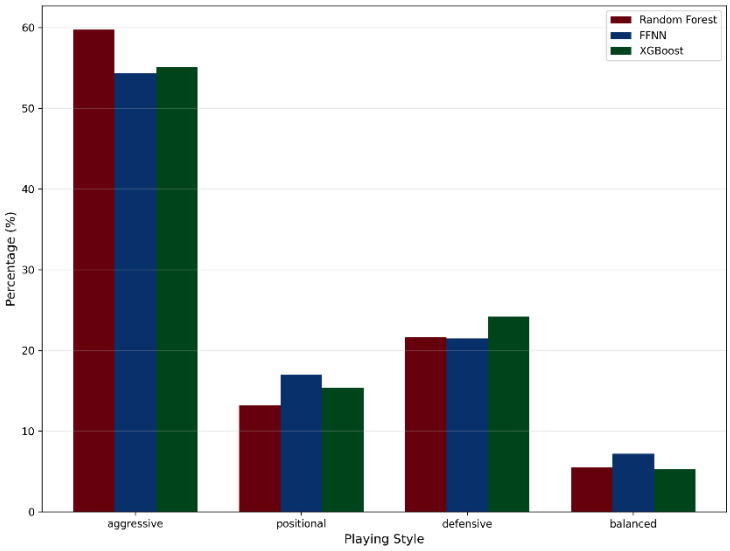
Some divergence exists between the two. The Random Forest assigns relatively high importance to blunders and total_moves - features that correlate with skill level as well as style. XGBoost weights counterattacks and positional_sacrifices most heavily, suggesting it picks up more style-specific signal with less skill-level noise. This may partly account for the accuracy advantage XGBoost achieves on the defensive and positional classes specifically.

5.5 Cross-Skill Generalisation

To address Objective 4, all three models were applied to the two held-out evaluation sets constructed from the 2016 Lichess database via eval_prediction.py: 500 game records from players rated below 1400 and 500 from players rated above 1600. The predicted class distributions are shown in Figures 5.1 and 5.2 and summarised in Table 5.2.

Style	High Elo	Low Elo
Aggressive	~39%	~56%
Positional	~29%	~15%
Defensive	~25%	~22%
Balanced	~7%	~6%

Table 5.2 Predicted class distributions (%) across Elo groups, averaged across all three models.



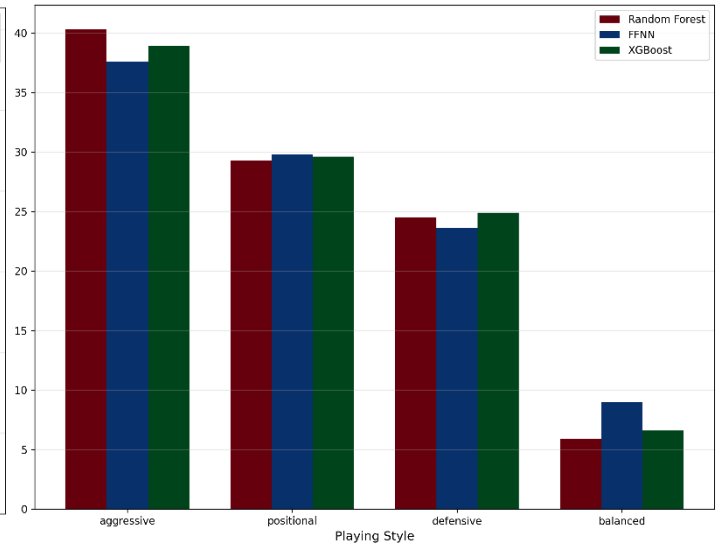


Figure 5.1 (Left) Predicted class distributions for the high-Elo evaluation set (<1399), showing the percentage of games assigned to each playing style by each of the three models.

Figure 5.2 (Right) Predicted class distributions for the low-Elo evaluation set (>1599), showing the percentage of games assigned to each playing style by each of the three models.

Model agreement - the proportion of samples where all three models produce the same prediction - was 86.7% for the high-Elo group and 85.9% for the low-Elo group, rates

comparable to in-distribution performance and consistent across two very different skill brackets. The most significant distributional shift is the rise in aggressive classification from approximately 39% at high Elo to 56% at low Elo, alongside a decline in positional from 29% to 15%. Lower-rated players engage in more frequent captures and piece exchanges, not necessarily by design but because opportunistic tactical play dominates at that skill level. Prophylactic moves and positional sacrifices require planning horizons that players below 1400 rarely exhibit, which accounts for the sharp drop in positional classification. Defensive and balanced distributions remain largely stable across both groups.

One limitation of this result is that at lower Elo levels, the aggressive label may partially reflect skill-level effects rather than deliberate stylistic preference. A player who captures frequently due to tactical opportunism registers the same feature signatures as an intentionally aggressive player. Because the feature extractor encodes observable behaviour rather than intent, this distinction cannot be resolved without subjective self-report data. Despite this, the models detect consistent structural differences between the two groups that are interpretable in chess terms, supporting the conclusion that the feature space generalises meaningfully across skill levels.

5.6 Limitations and Failure Cases

Four limitations of the system are identified. First, out-of-distribution games from historical periods cause systematic misclassification across all three models. Stockfish evaluates positional sacrifices from 19th-century games as blunders, producing centipawn loss values far outside the training distribution, which causes all three models to misclassify. This is an artefact of the feature extraction process rather than a model failure, and it motivates restricting the tool's intended use to contemporary games.

Second, the balanced class presents the greatest classification uncertainty across all models. This partly reflects limited training support - 275 samples compared to 1644 for aggressive - and partly the fact that balanced is defined residually, as the absence of strong features for any single style, making it harder to learn a coherent decision boundary.

Third, no external ground truth exists for the labelling methodology. Confidence in the labels rests on internal evidence: a roughly balanced class distribution, cross-model agreement of approximately 87%, and feature importance rankings that align with domain knowledge. A formal user study comparing model predictions against self-reported playing styles would provide stronger validation and remains an item of future work.

A fourth limitation specific to the deployed web interface is that analysis depth was reduced from 12 to 6 to accommodate server timeout constraints. This introduces a minor distribution shift between the features seen during training and those computed at inference time. The behavioural features (captures, checks, retreats, and similar) are less affected by this reduction. In practice the impact on classification output is expected to be small, but addressing it is a candidate for future work - alternative deployment approaches such as asynchronous processing or a more powerful server would allow full depth analysis.

5.7 Conclusions

This project set out to automatically classify chess playing styles from game data and make the results accessible through a web interface. The labelled dataset constructed in Stage 1 shows a balanced class distribution and high cross-model agreement, providing internal confidence in the validity of the labelling methodology despite the absence of an external benchmark.

The Random Forest (91.2%) outperformed the FFNN (89.8%) on the classification task, consistent with the established finding that tree-based models generalise better than neural networks on tabular data with heterogeneous features (Grinsztajn et al., 2022). XGBoost achieved the strongest overall performance at 96.9%, with its sequential boosting mechanism resolving the aggressive-defensive confusion that both other models exhibit. The convergence of feature importance rankings between RF and XGBoost - with captures_per_move, prophylactic_moves, and positional_sacrifices among the top features in both - provides evidence that the models are learning behaviourally meaningful patterns rather than noise.

Cross-skill evaluation on the 2016 Lichess data produced interpretable distributional differences between Elo brackets, with model agreement above 85% in both groups. The observable shift from positional to aggressive classification at lower Elo is consistent with known differences in how players at different skill levels approach the game and supports the conclusion that the feature space captures genuine stylistic signal across a wider population than the training data alone represents.

The web interface makes these classifications accessible to average players without requiring any technical knowledge, fulfilling the practical aim of the project.

5.8 Future Work

The most significant extension to the current system would be multi-game aggregation: classifying a player based on predictions across several recent games rather than a single one would produce a more stable output, particularly for the balanced class where single-game predictions are noisiest.

At higher Elo levels, incorporating opening repertoire as a feature becomes more defensible. It was deliberately excluded from the current system on the grounds that at intermediate Elo it reflects memorisation rather than stylistic intent, but above approximately 2000 repertoire choice more genuinely reflects preference and could improve positional and aggressive discrimination.

A sequence-based model - such as a transformer or LSTM operating directly over the move sequence rather than aggregate statistics - could capture temporal patterns that the current approach loses. A player who opens aggressively but consolidates positionally in the endgame would be classified by the current system on the aggregate balance of those behaviours, whereas a sequence model could represent the stylistic shift explicitly.

Finally, a user study asking rated players to self-report their playing style and comparing results against model predictions would address the most significant remaining validation gap. Even a modest study of 30 - 50 players would provide meaningful external evidence for the labelling methodology that internal metrics alone cannot supply.

List of References

Aniketh Kalagara. “Predictive Modelling of a Chess Player’s Style Using Machine Learning - NHSJS.” *NHSJS*, 9 Apr. 2024, nhsjs.com/2024/predictive-modelling-of-a-chess-players-style-using-machine-learning/.

Breiman, Leo. “Random Forests.” *Machine Learning*, vol. 45, no. 1, Oct. 2001, pp. 5–32.

Chen, Tianqi, and Carlos Guestrin. “XGBoost: A Scalable Tree Boosting System.” *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining - KDD ’16*, vol. 1, no. 1, 13 Aug. 2016, pp. 785–794, https://doi.org/10.1145/2939672.2939785.

Glickman, Mark. “Mark Glickman’s Research Page.” *Glicko.net*, 2016, www.glicko.net/research.html. Accessed 29 Apr. 2026.

Grinsztajn, Léo, et al. “Why Do Tree-Based Models Still Outperform Deep Learning on Tabular Data?” *ArXiv:2207.08815 [Cs, Stat]*, 18 July 2022, arxiv.org/abs/2207.08815.

Jayasekara, M.G.P.B. “Classification of Chess Games and Players by Styles Using Game Data.” *Cmb.ac.lk*, 2018, dl.ucsc.cmb.ac.lk/jspui/handle/123456789/4219, http://dl.ucsc.cmb.ac.lk/jspui/handle/123456789/4219. Accessed 10 Dec. 2025.

Kingma, Diederik P, and Jimmy Ba. “Adam: A Method for Stochastic Optimization.” *ArXiv*, 22 Dec. 2014, arxiv.org/abs/1412.6980.

McIlroy-Young, Reid, et al. “Aligning Superhuman AI with Human Behavior.” *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 6 July 2020, arxiv.org/pdf/2006.01855.pdf, https://doi.org/10.1145/3394486.3403219.

Pedregosa, Fabian, et al. “Scikit-Learn: Machine Learning in Python” *Journal of Machine Learning Research*, vol. 12, no. 12, 2011, pp. 2825–2830, www.jmlr.org/papers/volume12/pedregosa11a/pedregosa11a.pdf.

Silver, David, et al. “A General Reinforcement Learning Algorithm That Masters Chess, Shogi, and Go through Self-Play.” *Science*, vol. 362, no. 6419, 6 Dec. 2018, pp. 1140–1144, https://doi.org/10.1126/science.aar6404.

Appendix A
Self-appraisal

A.1 Critical self-evaluation

All five project objectives were delivered. A labelled dataset was constructed, three classifiers were trained and evaluated, cross-skill generalisation was assessed across two Elo brackets, and a web interface was built to make the system accessible to average players. The performance results - 91.2% for the Random Forest, 89.8% for the FFNN, and 96.9% for XGBoost - exceed chance-level performance and produce outputs that are interpretable in chess terms.

The most significant limitation is the absence of an external ground truth for the labelling methodology. Confidence in the labels rests on indirect evidence: a balanced class distribution, cross-model agreement of approximately 87%, and feature importance rankings that align with domain knowledge. A user study comparing model predictions against self-reported playing styles would have addressed this gap but was not feasible within the project timeline.

The balanced class presented ongoing difficulty. With only 275 samples compared to 1644 for aggressive, it was the weakest-performing class across all three models. Oversampling techniques such as further threshold adjustment during labelling could have improved this. Hyperparameter tuning was also limited to common practice defaults though the results achieved suggest the configurations were already close to optimal for this dataset.

A.2 Personal reflection and lessons learned

The most significant lesson was how much time data preprocessing would consume. I estimated the dataset construction phase would take roughly three weeks; in practice it took closer to six, mainly due to the iterative threshold calibration needed to achieve a reasonable class distribution. The initial labelling run produced 53% aggressive and 3.5% defensive labels, requiring a redesign of the defensive feature set and multiple full pipeline re-runs. This reduced the time available for model development. In retrospect, prototyping the labelling logic on a small sample before committing to a full run would have identified this problem much earlier.

A.3 Legal, social, ethical and professional issues

A.3.1 Legal issues

All external resources were used in compliance with their licence terms. The Lichess Open Database is released under Creative Commons CC0, imposing no restrictions on use. Stockfish is distributed under the GNU GPL v3; the project uses it as an external binary without modifying or redistributing its source code, which is compliant. The Python libraries - scikit-learn, PyTorch, XGBoost, and python-chess - are all distributed under permissive open-source licences permitting academic use.

The web interface does not store personally identifiable information. Submitted PGN data is processed in memory and discarded after the prediction is returned.

A.3.2 Social issues

The project was motivated by reducing an information imbalance: personalised style analysis has historically been available only to players who can afford coaching. Making it accessible through a web interface has a positive social dimension. One potential negative effect is that players might optimise for a label rather than genuine improvement - deliberately playing more aggressively to receive an aggressive classification, for example. The tool is intended to describe current tendencies, not prescribe how someone should play, and this is communicated in the interface.

A.3.3 Ethical issues

The primary ethical consideration is that labels are assigned algorithmically to human behaviour without participant consent or self-report validation. A player classified as defensive may have been responding to a specific difficult position rather than exhibiting a general tendency. The system observes behaviour rather than intent, and this distinction is disclosed in both the report and the interface. Lichess usernames present in raw PGN files are not used as features, stored, or surfaced to users.

A.3.4 Professional issues

The BCS Code of Conduct requires that computing professionals act in the public interest, work with due care, and represent their work honestly. The report explicitly acknowledges the absence of external ground truth and documents known failure modes rather than omitting them. The codebase is version-controlled, reproducible given a fixed random seed, and sufficiently documented to allow the pipeline to be re-run independently.

Appendix B
External Materials

Lichess Open Database. Training data was drawn from the 2015 Lichess Open Database, and cross-skill evaluation sets from the 2016 database. Both are released under Creative Commons CC0. All filtering, feature extraction, and labelling were implemented as part of this project.

Stockfish. Position evaluation was performed using Stockfish 16 (GNU GPL v3), interfaced via the python-chess library. No modifications were made to the Stockfish source code.

python-chess. Used to parse PGN files, traverse move sequences, and query board state during feature extraction. Distributed under the GNU GPL v3.

scikit-learn. Used to implement the Random Forest classifier, StandardScaler preprocessing, stratified splitting, and classification metrics. Distributed under the BSD 3-Clause licence.

PyTorch. Used to implement the feedforward neural network, training loop, and model serialisation. Distributed under the BSD licence.

XGBoost. Used to implement the gradient boosting classifier. Distributed under the Apache 2.0 licence.

All other code (the feature extraction pipeline, labelling algorithm, training scripts, evaluation scripts, and web interface) was written entirely by the author.